



Санкт-Петербургский государственный университет
Искусственный интеллект и наука о данных

MLP-Mixer и ConvMixer

Гришина Ксения Анатольевна, группа 23.Б16-мм

Доклад по дисциплине
«Алгоритмы обработки изображений»

Санкт-Петербург
2026

- CNN — де-факто стандарт в работе с изображениями
- ViT (2021) — новый популярный подход
- Вопрос: действительно ли сложные механизмы — свёртки или self-attention — необходимы?
- В рассматриваемых работах показано, что конкурентоспособные модели могут быть простыми:
 - 1 MLP-Mixer (2021)
 - 2 ConvMixer (2022)

MLP-Mixer

Идея

- Только MLP
- Без CNN и attention

Особенности

- Раздельная обработка:
 - ▶ пространства
 - ▶ признаков
- Используются простые матричные операции

Подход

- Изображение $\rightarrow$ патчи
- Channel-mixing MLP
- Token-mixing MLP

Интерпретация

- Аналогии с CNN:
 - ▶ смешивание каналов $\leftrightarrow 1 \times 1$ свёртки
 - ▶ смешивание токенов $\leftrightarrow$ глобальные свёртки
- $\text{Mixer} \subset \text{CNN}$, $\text{CNN} \not\subset \text{Mixer}$

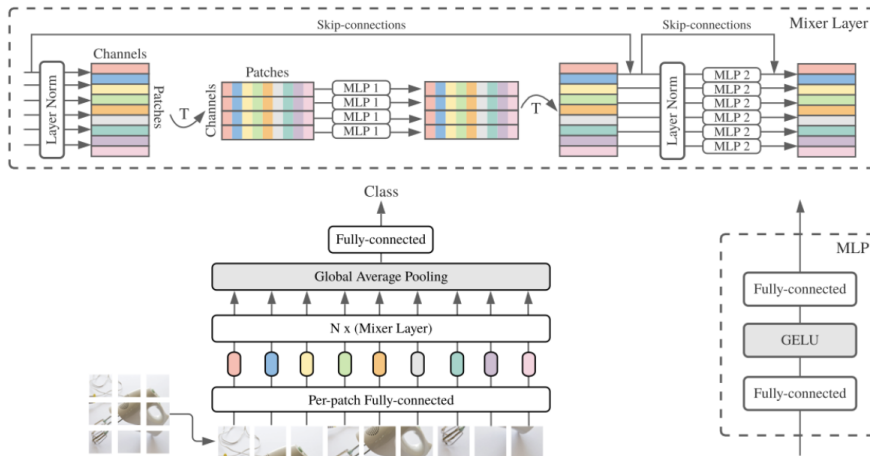


Рис.: Архитектура MLP-Mixer

MLP-Mixer: входное представление

- Изображение: $H \times W \times 3$
- Разбиение на S патчей $P \times P$, где $S = \frac{HW}{P^2}$
- Каждый патч проецируется в пространство скрытой размерности C :

$$x_i \in \mathbb{R}^{P^2 \cdot 3} \rightarrow z_i \in \mathbb{R}^C$$

- Итоговое матричное представление изображения:

$$X \in \mathbb{R}^{S \times C}$$

- Проход через Mixer слои = 2 блока:
 - ▶ Token-mixing
 - ▶ Channel-mixing

Token-mixing MLP

Обработка по пространству (между патчами)

- Фиксируем канал:

$$X_{:,j} \in \mathbb{R}^S$$

- ▶ Один MLP применяется ко всем каналам
- ▶ Учитывает зависимости между патчами

TokenMix

$$\text{TokenMix}(X) = X + W_2 \sigma(W_1 \cdot \text{LayerNorm}(X)), \text{ где}$$

$W_1 \in \mathbb{R}^{S \times D}$, $W_2 \in \mathbb{R}^{D \times S}$, σ — функция активации (GELU)

Channel-mixing MLP

Обработка по каналам (внутри патча)

- Фиксируем патч:

$$X_{i,:} \in \mathbb{R}^C$$

- ▶ Смешивание признаков
- ▶ Аналог 1×1 свёртки

ChannelMix

$$\text{ChannelMix}(U) = U + W_4 \sigma(W_3 \cdot \text{LayerNorm}(U)), \text{ где}$$

$$W_3 \in \mathbb{R}^{C \times D'}, W_4 \in \mathbb{R}^{D' \times C}$$

Изотропность

- Размер представления не меняется
- $S \times C$ на всех слоях

Глобальный контекст

- Полное receptive field
- Каждый слой видит все патчи

Эффективность

- Линейная сложность: $O(S)$
- Transformer: $O(S^2)$

Масштабируемость

- Лучше растёт с размером данных
- Подходит для больших изображений

Пример

```
from torch import nn
import torch.nn.functional as F

# MLP: Linear -> GELU -> Dropout -> Linear
class MLP(nn.Module):
    def __init__(self, dim, exp=2, drop=0.):
        super().__init__()
        self.fc1 = nn.Linear(dim, dim * exp)
        self.fc2 = nn.Linear(dim * exp, dim)
        self.drop = nn.Dropout(drop)

    def forward(self, x):
        return self.drop(self.fc2(self.drop(F.gelu(self.fc1(x)))))

# MixerLayer = TokenMixing + ChannelMixing
class MixerLayer(nn.Module):
    def __init__(self, n_patches, dim, exp=2, drop=0.):
        super().__init__()
        self.norm1, self.norm2 = nn.LayerNorm(dim), nn.LayerNorm(dim)
        self.token_mlp = MLP(n_patches, exp, drop) # mixing по патчам
        self.channel_mlp = MLP(dim, exp, drop) # mixing по каналам

    def forward(self, x): # x: [B, N, D]
        x = x + self.token_mlp(self.norm1(x).transpose(1, 2)).transpose(1, 2)
        x = x + self.channel_mlp(self.norm2(x))
        return x
```

```
# MLP Mixer: patching -> Mixer layers -> pooling -> classifier
class MLP Mixer(nn.Module):
    def __init__(self, img_size=256, patch=16, in_ch=3,
                 dim=128, depth=8, exp=2, n_classes=10, drop=0.):
        super().__init__()
        n_patches = (img_size // patch) ** 2
        # patch embedding через conv
        self.patcher = nn.Conv2d(in_ch, dim, patch, patch)
        # стек Mixer слоев
        self.mixer = nn.Sequential(*[
            MixerLayer(n_patches, dim, exp, drop)
            for _ in range(depth)
        ])
        self.head = nn.Linear(dim, n_classes)

    def forward(self, x):
        x = self.patcher(x) # [B, D, H', W']
        x = x.flatten(2).transpose(1, 2) # [B, N, D]
        x = self.mixer(x)
        return self.head(x.mean(1)) # pooling + классификация
```

Рис.: Реализация MLP Mixer

Рис.: Реализация MLP и MixerLayer

Метрики:

- Top-1 Accuracy
- скорость инференса
- стоимость обучения

Наборы данных:

- ImageNet-1k
- CIFAR-10/100
- Oxford Pets / Flowers
- VTAB-1k

Предобучены на:

- ImageNet-1k
- ImageNet-21k
- JFT-300M

Эксперимент 1. Основные результаты

	ImNet top-1	ReaL top-1	Avg 5 top-1	VTAB-1k 19 tasks	Throughput img/sec/core	TPUv3 core-days
Pre-trained on ImageNet-21k (public)						
• HaloNet [51]	85.8	—	—	—	120	0.10k
• Mixer-L/16	84.15	87.86	93.91	74.95	105	0.41k
• ViT-L/16 [14]	85.30	88.62	94.39	72.72	32	0.18k
• BiT-R152x4 [22]	85.39	—	94.04	70.64	26	0.94k
Pre-trained on JFT-300M (proprietary)						
• NFNet-F4+ [7]	89.2	—	—	—	46	1.86k
• Mixer-H/14	87.94	90.18	95.71	75.33	40	1.01k
• BiT-R152x4 [22]	87.54	90.54	95.33	76.29	26	9.90k
• ViT-H/14 [14]	88.55	90.72	95.97	77.63	15	2.30k
Pre-trained on unlabelled or weakly labelled data (proprietary)						
• MPL [35]	90.0	91.12	—	—	—	20.48k
• ALIGN [21]	88.64	—	—	79.99	15	14.82k

Рис.: Сравнение моделей (ImageNet / ReaL / Avg-5 / VTAB-1k)

Эксперимент 2. Влияние масштаба

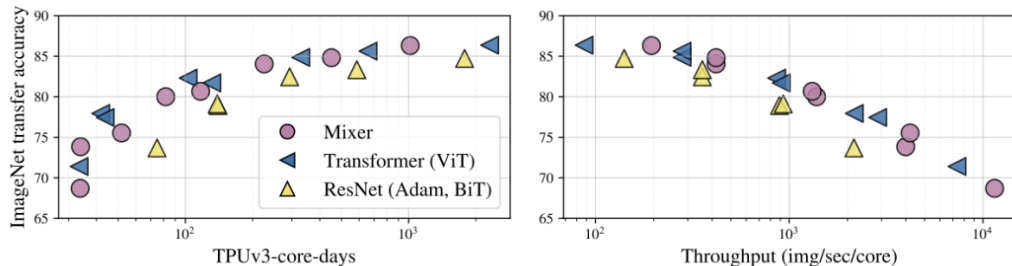


Рис.: Зависимость качества от вычислений и пропускной способности при разных масштабах

- Увеличение модели улучшает качество
- Эффект проявляется только при больших данных
- Сильная зависимость от масштаба обучения

Эксперимент 3. Инвариантность к перестановкам

- Проверка устойчивости к перемешиванию входа
 - Перестановка порядка патчей и пикселей в них
 - Полная перестановка всех пикселей изображения

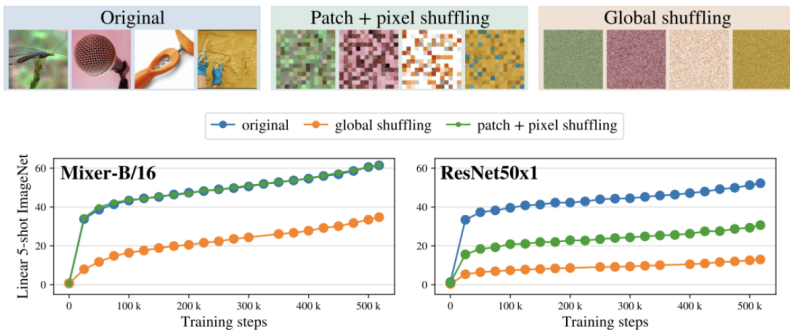


Рис.: Реакция моделей на перестановку входных данных

Вывод экспериментов

- ❶ Простая MLP-архитектура конкурентоспособна
- ❷ Критически важен масштаб данных
- ❸ Mixer = сильная data-driven модель

ConvMixer

- Статья: “Patches Are All You Need?”
- Вопрос: нужны ли attention и MLP?
- Гипотеза: важнее сам факт патчей
- Замена MLP → свёртки
- Сохраняется идея:
 - ▶ разделение пространства и каналов

ConvMixer: архитектура

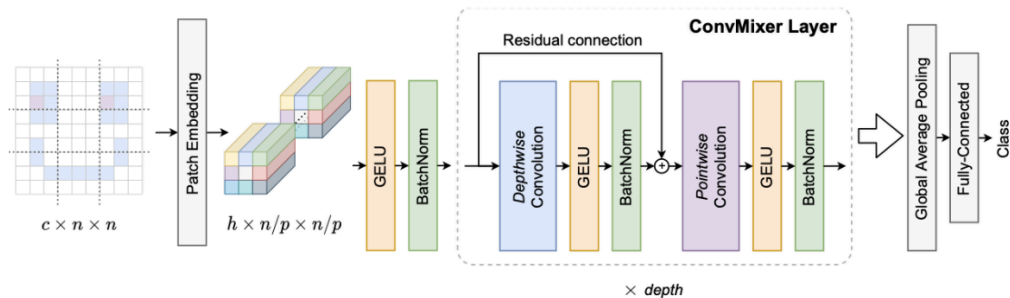


Рис.: Архитектура ConvMixer

- Embedding слой + стек Conv-блоков
- Сохраняется пространственная структура

Patch embedding через свёртку

$$z_0 = \text{BN}(\sigma(\text{Conv}_{c_{in} \rightarrow h}(X, p, p)))$$

- Kernel size = patch size = p
- Stride = patch size = p
- Одновременно:
 - ▶ разбиение на патчи
 - ▶ проекция в embedding

ConvMixer block

1. Depthwise convolution

- Пространственное смешивание
- Большое receptive field (7×7 , 9×9)

ConvDepthwise

$$z'_l = \text{BN}(\sigma(\text{ConvDepthwise}(z_{l-1}))) + z_{l-1}$$

2. Pointwise convolution (1×1)

- Смешивание каналов

ConvPointwise

$$z_{l+1} = \text{BN}(\sigma(\text{ConvPointwise}(z'_l)))$$

Разделение операций

- spatial mixing (depthwise conv)
- channel mixing (1×1 conv)

Стабилизация обучения

- Residual connections
- BatchNorm
- ReLU / GELU

Структура данных

- Сохранение пространственной структуры
- Нет “схлопывания” в вектор сразу

Выход

- Global pooling в конце
- Затем linear classifier

Пример и "фишка" статьи

```
def ConvMixer(h,d,k,p,n):  
    S,C,A=Sequential,Conv2d,lambda x:S(x,GELU(),BatchNorm2d(h))  
    R=type('',(S,),'forward':lambda s,x:s[0](x)+x}  
    return S(A(C(3,h,p,p)),*[S(R(A(C(h,h,k,groups=h,padding=k//2))),A(C(h,h,1))) for  
    i in range(d)],AdaptiveAvgPool2d(1),Flatten(),Linear(h,n))
```

Рис.: Реализация ConvMixer (очень компактная)

- Модель уместается в несколько строк кода
- Основной текст статьи — всего 4.5 страницы

Эксперимент 1. Основные результаты (ImageNet)

Основные результаты

- ImageNet: 80–81% Top-1
- Сопоставимо с ResNet и ViT
- Без настройки гиперпараметров
- Хороший результат при ограниченных ресурсах

Вывод

- Простая модель даёт конкурентное качество

Comparison with other simple models trained on ImageNet-1k only with input size 224.							
Network	Patch Size	Kernel Size	# Params ($\times 10^6$)	Throughput (img/sec)	Act. Fn.	# Epochs	ImNet top-1 (%)
ConvMixer-1536/20★	7	9	51.6	134	G	150	82.20
ConvMixer-1536/20●	7	9	51.6	134	G	150	81.37
ConvMixer-1536/20★	7	3	49.4	246	G	150	81.60
ConvMixer-1536/20	7	3	49.4	246	G	150	80.43
ConvMixer-1536/20	14	9	52.3	538	G	150	78.92
ConvMixer-1536/24★	14	9	62.3	447	G	150	80.21
ConvMixer-768/32●	7	7	21.1	206	R	300	80.16
ConvMixer-1024/16	7	9	19.4	244	G	100	79.45
ConvMixer-1024/12	7	8	14.6	358	G	90	77.75
ConvMixer-512/16	7	8	5.4	599	G	90	73.76
ConvMixer-512/12●	7	8	4.2	798	G	90	72.59
ConvMixer-768/32	14	3	20.2	1235	R	300	74.93
ConvMixer-1024/20●	14	9	24.4	750	G	150	76.94
ResNet-152●	–	3	60.2	828	R	150	79.64
ResNet-101●	–	3	44.6	1187	R	150	78.33
ResNet-50	–	3	25.6	1739	R	150	76.32
DeiT-B†	7	–	86.7	83	G	–	–
DeiT-S†	7	–	22.1	174	G	–	–
DeiT-Ti†	7	–	5.7	336	G	–	–
DeiT-B●	16	–	86	792	G	300	81.8
DeiT-S●	16	–	22	1610	G	300	79.8
DeiT-Ti●	16	–	5.7	2603	G	300	72.2
ResMLP-S12/8●	8	–	22.1	872	G	400	79.1
ResMLP-B24/8●	8	–	129	181	G	400	81.0
ResMLP-B24	16	–	116	1597	G	400	81.0
Swin-S●	4	–	50	576	G	300	83.0
Swin-T●	4	–	29	878	G	300	81.3
ViT-B/16●	16	–	86	789	G	300	77.9
Mixer-B/16●	16	–	59	1025	G	300	76.44
Isotropic MobileNetv3●	8	3	20	–	R	–	80.6
Isotropic MobileNetv3●	16	3	20	–	R	–	77.6

Эксперимент 2. Анализ гиперпараметров (CIFAR-10)

Гиперпараметры

- Depthwise kernel size:
 - ▶ больше ядро → лучше качество
 - ▶ ключевую роль играет receptive field
- Patch size:
 - ▶ больше патч → хуже результат
 - ▶ потеря деталей
- Depth / width:
 - ▶ улучшает качество
 - ▶ wide модели сходятся быстрее

CIFAR-10

- >96% accuracy
- Малое число параметров
- Сильный эффект аугментаций:
 - ▶ RandAugment
 - ▶ Mixup
 - ▶ CutMix

Рассмотренные архитектуры показывают, что высокая точность достигается не только за счёт сложных механизмов (attention или сложные CNN), но и за счёт правильной организации обработки данных.

Основные выводы:

- MLP-Mixer требует больших данных, но остаётся максимально простой
- ConvMixer заменяет MLP на свёртки, сохраняя идею патчей

Список литературы

- 1 Tolstikhin I. O. et al. Mlp-mixer: An all-mlp architecture for vision //Advances in neural information processing systems. – 2021. – Т. 34. – С. 24261-24272.
- 2 Trockman A., Kolter J. Z. Patches are all you need? //arXiv preprint arXiv:2201.09792. – 2022.